



WANDA

Where Are you Now Detection App

Aplicación Android de rastreo inteligente del hogar con lista de la compra

Asignatura: Sistemad Agenti

Plataforma: Android (API 26–34)

Lenguaje: Kotlin

Versión: 1.0

Arquitectura: MVVM

Autores: Enrique Robeles
Juan Felix Noblejas Torroba
Alex Rodriguez
Bernardo Gómez

Índice de contenidos

1. Descripción del proyecto
2. WANDA como agente reactivo
3. Funcionalidades principales
4. Arquitectura del proyecto
5. Stack tecnológico
6. Descripción de componentes clave
7. Base de datos (Room / SQLite)
8. Pantallas de la aplicación
9. Permisos requeridos
10. Cómo compilar y ejecutar
11. Guía de uso paso a paso
12. Configuración avanzada
13. Decisiones de diseño técnico
14. Solución de problemas

1. Descripción del proyecto

WANDA (*Where Are you Now Detection App*) es una aplicación Android desarrollada en Kotlin para la asignatura de **Sistemi ad Agenti**. Su objetivo principal es demostrar, de forma práctica, los principios de

un agente reactivo situado en un entorno físico real: el domicilio del usuario.

La aplicación actúa como un **agente de contexto doméstico**. Esto significa que monitoriza continuamente la posición GPS del dispositivo y toma decisiones automáticas en función de si el usuario se encuentra dentro o fuera de una zona geográfica definida, llamada *geovalla* (geofence). Cuando el usuario cruza el perímetro de esa zona —ya sea al salir o al llegar a casa— WANDA reacciona de inmediato: registra el evento, lo almacena en una base de datos local y notifica al usuario a través de una notificación push.

Como funcionalidad complementaria, la aplicación integra una **lista de la compra persistente** que el usuario puede gestionar desde cualquier pantalla. Aunque esta característica no forma parte del núcleo del agente, enriquece la propuesta de valor de la app como herramienta de apoyo al hogar.

El proyecto está estructurado siguiendo la arquitectura **MVVM** (Model-View-ViewModel) recomendada por Google para Android, lo que garantiza una separación clara de responsabilidades, código mantenible y una experiencia de usuario fluida y reactiva.

2. WANDA como agente reactivo

En el contexto de la asignatura, un **agente** es una entidad capaz de percibir su entorno a través de sensores y actuar sobre él mediante efectores, de forma autónoma y continuada. WANDA es un ejemplo concreto de un **agente reactivo basado en sensores** que opera sobre el entorno físico del usuario.

El ciclo percepción–decisión–acción

El comportamiento de WANDA sigue el ciclo clásico de un agente reactivo:



El agente es **reactivo** porque no planifica: responde directamente a estímulos del entorno sin mantener un modelo interno del mundo ni ejecutar razonamiento deliberativo. En cuanto el sistema detecta una transición de geovalla, la respuesta es inmediata y determinista.

Autonomía y persistencia

Una propiedad clave de los agentes es la **autonomía**: la capacidad de operar sin intervención continua del usuario. WANDA la implementa de dos formas:

- **Foreground Service**: un servicio Android que se ejecuta en primer plano incluso cuando la aplicación está cerrada o el dispositivo en reposo. Esto garantiza que el agente nunca se detiene mientras el rastreo esté activado.
- **BootReceiver**: si el dispositivo se apaga o reinicia, el agente se reactiva automáticamente al arrancar, restaurando la geovalla sin que el usuario tenga que intervenir.

Entorno, sensores y efectores

Componente del agente	Implementación en WANDA
Entorno	Mundo físico: la posición geográfica del usuario
Sensor	GPS del dispositivo + Google Play Services Location
Función de percepción	Geofencing API: detecta transiciones de zona
Estado interno	PreferencesManager (coordenadas del hogar, radio, estado activo)
Efector — almacenamiento	Room DB: guarda el evento como GpsRecord
Efector — comunicación	NotificationManager: notificación push al usuario
Efector — UI	LiveData: actualiza la pantalla de inicio en tiempo real

3. Funcionalidades principales

WANDA agrupa sus capacidades en dos bloques: el núcleo del agente de localización y la funcionalidad auxiliar de lista de la compra. A continuación se describe cada funcionalidad con detalle.

Establecer domicilio

El usuario puede definir su hogar de dos maneras. La primera, y más cómoda, consiste en pulsar "Establecer casa aquí": la app obtiene la posición GPS actual del dispositivo y la guarda como referencia del domicilio. La segunda opción permite introducir manualmente latitud, longitud y radio mediante el formulario de edición, útil cuando el usuario quiere definir una ubicación diferente a donde se encuentra físicamente (por ejemplo, la ubicación de un familiar).

Geovalla configurable

Una *geovalla* es una región geográfica virtual definida por un centro (coordenadas del hogar) y un radio en metros. WANDA permite ajustar ese radio entre **50 y 5.000 metros**, con un valor por defecto de 100 metros. Un radio pequeño ofrece detección más precisa pero puede generar falsas alarmas si el GPS tiene deriva; un radio mayor es más tolerante pero puede activarse antes de llegar realmente a casa.

Rastreo en segundo plano

Una vez activado el rastreo, WANDA lanza un *Foreground Service* que mantiene la geovalla registrada en Google Play Services de forma permanente. El servicio muestra una notificación persistente en la barra de estado ("WANDA – Rastreo activo") para cumplir con los requisitos de Android y para que el usuario siempre sepa que la monitorización está en curso. Este servicio sigue activo aunque el usuario cierre la aplicación o apague la pantalla.

Detección de entrada y salida

Cuando el dispositivo cruza el perímetro de la geovalla en cualquier dirección, Google Play Services genera un evento de transición. WANDA captura ese evento y, en milisegundos, realiza tres acciones de forma simultánea: guarda un registro en la base de datos con la hora exacta y las coordenadas del cruce, envía una notificación push al usuario con un mensaje claro ("Has llegado a casa" o "Has salido de casa"), y actualiza el estado mostrado en la pantalla de inicio.

Historial de eventos

Todos los eventos de entrada y salida quedan almacenados de forma persistente en la base de datos local. El historial muestra cada evento con su tipo (ENTRADA / SALIDA), fecha y hora formateadas, y las coordenadas GPS exactas del cruce. La lista se actualiza en tiempo real mediante LiveData: en cuanto llega un nuevo evento, aparece automáticamente sin necesidad de recargar la pantalla.

Lista de la compra

La lista de la compra es una funcionalidad auxiliar que permite al usuario gestionar artículos pendientes de comprar. Cada artículo puede marcarse como comprado con un simple toque (se muestra tachado visualmente), y existe un botón para eliminar todos los artículos completados de una vez. Los datos persisten en la base de datos local, de forma que la lista sobrevive al cierre de la aplicación y a reinicios del dispositivo.

Persistencia tras reinicios

Si el dispositivo se reinicia mientras el rastreo estaba activo, el *BootReceiver* intercepta el evento del sistema `BOOT_COMPLETED` y relanza automáticamente el servicio de localización. El usuario no necesita abrir la app ni pulsar ningún botón: el agente retoma su operación en cuanto el sistema está disponible.

Funcionalidad	Descripción
Establecer casa	Define el domicilio con GPS automático o coordenadas manuales
Geovalla configurable	Radio ajustable entre 50 y 5.000 metros alrededor del domicilio
Rastreo en segundo plano	Foreground Service activo incluso con la app cerrada
Detección entrada/salida	Notificación push automática al cruzar el perímetro de la geovalla
Historial de eventos	Registro persistente de entradas/salidas con fecha, hora y coordenadas
Lista de la compra	Lista interactiva con artículos marcables como comprados
Persistencia en reinicios	El rastreo se reactiva automáticamente al encender el dispositivo

4. Arquitectura del proyecto

WANDA sigue el patrón **MVVM** (Model-View-ViewModel) recomendado por Google para aplicaciones Android modernas. Este patrón divide la aplicación en tres capas con responsabilidades claramente separadas, lo que facilita el mantenimiento, las pruebas y la escalabilidad del código.

Las tres capas de MVVM

Model — Capa de Datos

Gestiona toda la lógica de acceso y almacenamiento de datos. Incluye la base de datos Room, los DAOs, las entidades y los repositorios. No conoce nada de la UI.

ViewModel — Capa de Dominio

Contiene la lógica de negocio y expone datos a la UI mediante LiveData. Sobrevive a rotaciones de pantalla. Llama a los repositorios y transforma los datos para la vista.

View — Capa de Presentación

Fragments y layouts XML. Solo observa el ViewModel y actualiza la interfaz. No accede nunca

directamente a la base de datos ni al GPS.

Esta separación es especialmente importante en WANDA porque el agente de localización opera de forma independiente a la interfaz gráfica. El servicio de rastreo (`LocationTrackingService`) escribe en la base de datos, y la UI se actualiza sola gracias a los observadores LiveData, sin necesidad de que ambas capas se comuniquen directamente.

Estructura de directorios

```
com.wanda.app/
├── MainActivity.kt           # Actividad principal + Bottom Navigation
├── WandaApplication.kt      # Clase Application: inicializa DB y repositorios
├── data/                    # — CAPA DE DATOS —
│   ├── database/
│   │   ├── WandaDatabase.kt # Base de datos Room (SQLite)
│   │   └── dao/
│   │       ├── ShoppingDao.kt # Operaciones CRUD lista de la compra
│   │       └── GpsRecordDao.kt # Operaciones CRUD historial GPS
│   ├── entity/
│   │   ├── ShoppingItem.kt   # Entidad: ítem de la lista
│   │   └── GpsRecord.kt     # Entidad: registro de evento GPS
│   └── repository/
│       ├── ShoppingRepository.kt
│       └── GpsRepository.kt
├── geofencing/              # — CAPA DE GEOVALLAS Y RASTREO —
│   ├── GeofenceHelper.kt    # Gestión de la API de Geofencing de Google
│   ├── GeofenceBroadcastReceiver.kt # Receptor de eventos de geovalla
│   ├── LocationTrackingService.kt # Servicio en primer plano (foreground)
│   └── BootReceiver.kt      # Receptor de arranque del dispositivo
├── ui/                      # — CAPA DE PRESENTACIÓN (MVVM) —
│   ├── home/ → HomeFragment + HomeViewModel
│   ├── shopping/ → ShoppingFragment + ShoppingViewModel + ShoppingAdapter
│   └── history/ → HistoryFragment + HistoryViewModel + HistoryAdapter
└── utils/
    ├── PreferencesManager.kt # Persistencia en SharedPreferences
    └── PermissionHelper.kt   # Comprobación de permisos
```

Flujo de datos completo

1. El usuario pulsa "Iniciar rastreo" en HomeFragment | ▼ HomeViewModel — lanza —> LocationTrackingService (Foreground Service) | ▼ GeofenceHelper — registra geovalla en —> Google Play Services 2. El usuario sale de casa (o llega) | ▼ Google

Play Services — broadcast —> GeofenceBroadcastReceiver | 

▼ ▼ GpsRepository NotificationManager → inserta GpsRecord en DB → notificación push al usuario | ▼ Room / LiveData → HistoryFragment y HomeFragment se actualizan solos

5. Stack tecnológico

El stack de WANDA está compuesto exclusivamente por tecnologías oficiales del ecosistema Android, lo que garantiza compatibilidad a largo plazo, buena documentación y soporte activo de la comunidad.

Tecnología	Versión	Uso en el proyecto
Kotlin	1.x	Lenguaje principal. Más conciso y seguro que Java, con soporte nativo para corrutinas.
Android SDK	API 26–34	Plataforma objetivo. API 26 como mínimo garantiza compatibilidad amplia; 34 como target para usar las últimas APIs.
AndroidX Navigation	2.7.7	Gestión de navegación entre fragmentos con soporte para Bottom Navigation Bar.
Room (SQLite)	2.6.1	ORM oficial de Android. Abstrae SQLite con seguridad de tipos en tiempo de compilación.
Google Play Services Location	21.2.0	Provee FusedLocationProvider (GPS eficiente) y la API de Geofencing.
ViewModel + LiveData	2.7.0	Columna vertebral del patrón MVVM. LiveData es observable y aware del ciclo de vida.
Kotlin Coroutines	1.7.3	Concurrencia asíncrona sin callbacks. Usado para operaciones de I/O a la base de datos.
ViewBinding	—	Acceso seguro y tipado a las vistas XML. Elimina los <code>findViewById</code> y los nulos inesperados.
KSP	—	Procesador de anotaciones de Room. Más rápido que KAPT en compilación incremental.
Material Design	1.11.0	Componentes UI estándar (botones, diálogos, FAB, colores) que siguen las guías de Google.

6. Descripción de componentes clave

WandaApplication

`WandaApplication` es la clase `Application` personalizada de la app. Se instancia antes que cualquier Activity o Fragment y actúa como **contenedor de dependencias manual**: inicializa la base de datos Room, los repositorios y el `PreferencesManager`, y crea los canales de notificación necesarios para Android 8+. Al exponer estas instancias como propiedades de la aplicación, cualquier componente puede acceder a ellas sin necesidad de un framework de inyección de dependencias.

PreferencesManager

Wrapper sobre `SharedPreferences` que encapsula toda la configuración persistente del agente. Almacena cuatro valores críticos:

- `homeLatitude` / `homeLongitude`: coordenadas decimales del domicilio.
- `geofenceRadius`: radio de la geovalla en metros (por defecto: 100 m).
- `isTrackingEnabled`: flag que indica si el rastreo está activo. Lo usa el `BootReceiver` para saber si debe relanzar el servicio.
- `isHomeSet`: flag que controla si se ha definido un domicilio. Sin él, el botón de iniciar rastreo permanece deshabilitado.

LocationTrackingService

Servicio Android de tipo `foregroundServiceType="location"` que representa el núcleo del agente. Al iniciarse, muestra obligatoriamente una notificación persistente en la barra de estado (requisito de Android para servicios en primer plano que acceden a la ubicación), lee las coordenadas del hogar desde `PreferencesManager` y delega en `GeofenceHelper` el registro de la geovalla en Google Play Services.

Devuelve `START_STICKY` para que el sistema operativo lo relance automáticamente si lo termina por falta de memoria. Al destruirse, elimina la geovalla del sistema para evitar notificaciones huérfanas.

GeofenceHelper

Encapsula toda la interacción con la `GeofencingClient` de Google Play Services. Crea una geovalla circular con el ID `"HOME_GEOFENCE"`, radio configurable y expiración `NEVER_EXPIRE` (la geovalla es permanente hasta que se elimine explícitamente). Detecta los dos tipos de transición: `GEOFENCE_TRANSITION_ENTER` y `GEOFENCE_TRANSITION_EXIT`.

El evento de transición se entrega a la app mediante un `PendingIntent` que apunta a `GeofenceBroadcastReceiver`. En Android 12 y versiones posteriores, este Intent debe incluir una acción explícita (`com.wanda.app.ACTION_GEOFENCE_EVENT`) porque los BroadcastReceivers implícitos están bloqueados por razones de seguridad del sistema operativo.

Nota técnica (Android 12+): El PendingIntent del Geofencing debe declarar la acción explícita `com.wanda.app.ACTION_GEOFENCE_EVENT`, y el receiver debe estar marcado como `exported="true"` en el AndroidManifest porque Google Play Services es una aplicación externa que envía el broadcast. Sin ambas condiciones, el receiver nunca se activa.

GeofenceBroadcastReceiver

Receptor que se activa cuando Google Play Services detecta una transición de geovalla. Dado que los `BroadcastReceiver` tienen un límite de ejecución de unos 10 segundos, usa `goAsync()` para obtener tiempo adicional y poder completar la operación de escritura en base de datos sin que Android mate el proceso. El flujo interno es:

- 1 Extrae el tipo de transición del `GeofencingEvent` recibido.
- 2 Lanza una corrutina en el dispatcher de IO para insertar el `GpsRecord` en Room.
- 3 Construye y muestra la notificación push con el mensaje adecuado.
- 4 Llama a `pendingResult.finish()` para liberar el proceso.

BootReceiver

Receptor del evento de sistema `BOOT_COMPLETED`. Al arrancar el dispositivo, consulta el flag `isTrackingEnabled` en `PreferencesManager`. Si estaba activo, lanza `LocationTrackingService` para restaurar la geovalla de forma transparente al usuario. Este mecanismo implementa la propiedad de **continuidad del agente** frente a interrupciones del entorno de ejecución.

7. Base de datos (Room / SQLite)

WANDA usa **Room** como capa de abstracción sobre SQLite. Room garantiza seguridad de tipos en tiempo de compilación (los errores SQL se detectan al compilar, no en ejecución), integración nativa con Coroutines y LiveData, y un API limpia mediante anotaciones Kotlin.

La base de datos contiene dos tablas independientes, cada una con su propio propósito:

ShoppingItem — tabla `shopping_items`

Almacena los artículos de la lista de la compra. Cada registro tiene un nombre, un indicador booleano de si ya ha sido comprado, y una marca de tiempo de creación para mantener el orden de inserción.

```
@Entity(tableName = "shopping_items")
data class ShoppingItem(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
```

```

    val name: String,
    val isChecked: Boolean = false,        // marcado como comprado
    val createdAt: Long = System.currentTimeMillis()
)

```

GpsRecord — tabla `gps_records`

Almacena cada evento de cruce de geovalla. El campo `type` toma el valor `"ENTRADA"` o `"SALIDA"` según la dirección del cruce. Las coordenadas registradas corresponden a la posición del dispositivo en el momento exacto del evento.

```

@Entity(tableName = "gps_records")
data class GpsRecord(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val type: String,                // "ENTRADA" o "SALIDA"
    val timestamp: Long = System.currentTimeMillis(),
    val latitude: Double,
    val longitude: Double
)

```

Acceso a datos mediante DAOs

Cada tabla tiene su propio DAO (*Data Access Object*) que define las operaciones disponibles. Los métodos marcados con `suspend` son funciones de corrutina que deben ejecutarse fuera del hilo principal; los que devuelven `LiveData` se observan de forma reactiva desde la UI.

DAO	Operación	Descripción
GpsRecordDao	<code>getAllRecords()</code>	Devuelve LiveData con todos los eventos, ordenados por fecha descendente
GpsRecordDao	<code>insertRecord(record)</code>	Inserta un nuevo evento de geovalla (suspend)
GpsRecordDao	<code>getLastRecord()</code>	Devuelve el evento más reciente para mostrarlo en la pantalla de inicio
ShoppingDao	<code>getAllItems()</code>	Devuelve LiveData con todos los artículos, ordenados por fecha de creación
ShoppingDao	<code>insertItem(item)</code>	Añade un nuevo artículo a la lista (suspend)
ShoppingDao	<code>updateItem(item)</code>	Actualiza el estado checked/unchecked de un artículo (suspend)

DAO	Operación	Descripción
ShoppingDao	<code>deleteCheckedItems()</code>	Elimina todos los artículos marcados como comprados (suspend)

8. Pantallas de la aplicación

La navegación de WANDA se basa en una **Bottom Navigation Bar** con tres destinos. La navegación está gestionada por el componente Navigation de Jetpack, que mantiene el estado de cada fragmento al cambiar de pestaña.

Inicio (HomeFragment)

Es la pantalla principal y el panel de control del agente. Muestra en tiempo real el **estado actual** del usuario ("Estás en casa" / "Has salido"), actualizándose automáticamente cada vez que llega un nuevo evento de geovalla gracias a un observador LiveData.

Desde esta pantalla el usuario puede:

- **Establecer casa aquí:** captura la posición GPS actual y la guarda como domicilio.
- **Editar casa:** abre un diálogo para modificar manualmente latitud, longitud y radio (entre 50 y 5.000 m). Si el rastreo estaba activo, el servicio se reinicia automáticamente para aplicar los nuevos valores.
- **Borrar casa:** elimina las coordenadas del domicilio y detiene el rastreo.
- **Iniciar / Detener rastreo:** activa o desactiva el `LocationTrackingService`. El botón solo está habilitado si hay un domicilio configurado.

La pantalla también muestra el último evento registrado (tipo, fecha y hora) y el estado actual del rastreo (activo/inactivo) con las coordenadas y el radio de la geovalla.

Lista de la compra (ShoppingFragment)

Pantalla de gestión de la lista del supermercado. Presenta un `RecyclerView` reactivo que se actualiza automáticamente cuando se añaden o modifican artículos. Los elementos marcados como comprados aparecen tachados visualmente para diferenciarlos de los pendientes.

- Botón flotante (FAB "+"): abre un diálogo para escribir el nombre del nuevo artículo.
- Toque sobre un artículo: alterna su estado comprado/pendiente.
- Botón "Eliminar completados": visible únicamente cuando hay artículos marcados. Elimina todos de una vez.

Historial (HistoryFragment)

Muestra la lista cronológica completa de todos los eventos de geovalla registrados por el agente. Cada entrada indica el tipo de transición (ENTRADA o SALIDA), la fecha y hora formateadas, y las coordenadas GPS exactas del cruce. Si aún no hay eventos, se muestra un mensaje informativo en pantalla vacía.

Al igual que las demás pantallas, el historial se actualiza en tiempo real: cuando llega un nuevo evento mientras el usuario está viendo esta pantalla, la nueva entrada aparece automáticamente en la parte superior sin necesidad de interacción del usuario.

9. Permisos requeridos

WANDA necesita acceso a un conjunto específico de permisos del sistema Android. Algunos se conceden automáticamente al instalar la app; otros requieren confirmación explícita del usuario en tiempo de ejecución. A continuación se detalla cada permiso y por qué es imprescindible.

Permiso	Tipo	Motivo
ACCESS_FINE_LOCATION	Runtime	Precisión GPS completa. Necesaria para que la detección de geovalla sea fiable a escalas de metros.
ACCESS_COARSE_LOCATION	Runtime	Ubicación aproximada mediante red/WiFi. Actúa como fallback si el GPS tarda en obtener señal.
ACCESS_BACKGROUND_LOCATION	Runtime especial	Acceso a la ubicación cuando la app está en segundo plano (Android 10+). Imprescindible para que la geovalla funcione con la app cerrada.
FOREGROUND_SERVICE	Normal	Permite ejecutar el servicio de rastreo en primer plano con notificación visible.
FOREGROUND_SERVICE_LOCATION	Normal	Declara explícitamente que el foreground service accede a la ubicación (obligatorio en Android 14+).
POST_NOTIFICATIONS	Runtime	Mostrar notificaciones push de entrada/salida y la notificación persistente del servicio (Android 13+).
RECEIVE_BOOT_COMPLETED	Normal	Escuchar el evento de arranque del dispositivo para reactivar el rastreo automáticamente.

Importante: El permiso ACCESS_BACKGROUND_LOCATION debe concederse manualmente desde los ajustes del sistema: Ajustes → Aplicaciones → WANDA → Permisos → Ubicación → "Permitir todo el tiempo". Android

no permite solicitarlo junto con los demás permisos de ubicación en el mismo diálogo. Sin este permiso, la geovalla solo funciona mientras la app está en primer plano.

10. Cómo compilar y ejecutar

Requisitos previos

- Android Studio Hedgehog (2023.1.1) o superior
- JDK 17 (incluido en las versiones recientes de Android Studio)
- SDK de Android API 26 o superior instalado
- Dispositivo físico Android con Google Play Services (ver advertencia abajo)

Pasos para compilar

- 1 Abre Android Studio y selecciona **File** → **Open**. Navega hasta la carpeta `[ruta]/Ad-agenti-main/WANDA/` y selecciónala como proyecto raíz.
- 2 Una vez abierto el proyecto, Android Studio detectará el archivo `build.gradle.kts` y pedirá sincronizar. Acepta con **File** → **Sync Project with Gradle Files**. Espera a que Gradle descargue todas las dependencias (requiere conexión a internet en la primera ejecución).
- 3 Conecta un dispositivo Android físico mediante USB con **depuración USB activada** (Ajustes → Opciones de desarrollador → Depuración USB).
- 4 Selecciona tu dispositivo en el menú desplegable de la barra de herramientas y pulsa el botón **Run** (triángulo verde) o usa el atajo **Shift+F10**.

Advertencia: El Geofencing de Google Play Services **no funciona de forma fiable en emuladores**. La API requiere GPS real y acceso a los servicios de Google. Para probar las funcionalidades de geovalla es imprescindible usar un dispositivo físico. En emulador solo funcionarán las pantallas de UI estática y la lista de la compra.

11. Guía de uso paso a paso

Primera configuración

- 1 Abre WANDA. La app solicitará los permisos de ubicación (`FINE`, `COARSE` y `POST_NOTIFICATIONS`). Concédelos todos.

- 2 En Android 10 o superior, ve a Ajustes → Aplicaciones → WANDA → Permisos → Ubicación y selecciona "**Permitir todo el tiempo**". Sin esto el rastreo en segundo plano no funcionará.
- 3 Desde tu domicilio real, pulsa "**Establecer casa aquí**". La app tomará tu posición GPS actual y la guardará como referencia del hogar. Verás las coordenadas y el radio por defecto (100 m) en la pantalla de inicio.
- 4 Opcionalmente, pulsa "**Editar casa**" para ajustar el radio. Si vives en un edificio grande o zona densa, un radio mayor (200-300 m) reduce falsas alarmas.
- 5 Pulsa "**Iniciar rastreo**". Aparecerá la notificación persistente "WANDA – Rastreo activo" en la barra de estado. El agente ya está operativo.

Uso diario

- Al **salir de casa** recibirás la notificación "Has salido de casa" y el evento quedará registrado en el historial.
- Al **llegar a casa** recibirás "Has llegado a casa". La pantalla de inicio mostrará "Estás en casa".
- El rastreo continúa aunque cierres la aplicación, apagues la pantalla o reinicies el dispositivo.
- Para **detener el rastreo** temporalmente, abre WANDA y pulsa "Detener rastreo".

Uso de la lista de la compra

- Ve a la pestaña del carrito en la barra inferior.
- Pulsa el botón "+" (FAB) para añadir un artículo nuevo.
- Toca cualquier artículo para marcarlo como comprado (aparecerá tachado).
- Usa "**Eliminar completados**" para limpiar los artículos ya adquiridos.

12. Configuración avanzada

Modificar el radio de la geovalla

El radio determina a qué distancia del centro del domicilio se activa la detección. Valores recomendados según el contexto:

- **50–100 m:** ideal para casas unifamiliares o cuando se quiere detección muy precisa.
- **150–300 m:** recomendado para zonas urbanas densas donde el GPS puede tener deriva.
- **500–5.000 m:** útil para definir zonas más amplias (barrio, polígono, etc.).

Para modificarlo: pulsa "**Editar casa**" en la pantalla de inicio y ajusta el slider de radio. Si el rastreo estaba activo, el servicio se reinicia automáticamente para registrar la nueva geovalla con el radio actualizado.

Cambiar la ubicación del hogar

Hay dos métodos:

- **Opción 1:** Pulsa "Editar casa" e introduce manualmente las nuevas coordenadas en formato decimal (ej: 40.416775, -3.703790).
- **Opción 2:** Pulsa "Borrar casa" para eliminar el domicilio actual, desplázate físicamente a la nueva ubicación y pulsa "Establecer casa aquí".

13. Decisiones de diseño técnico

Durante el desarrollo de WANDA se tomaron varias decisiones de diseño que no son obvias a primera vista pero que son necesarias para que la aplicación funcione correctamente en el ecosistema Android moderno. Esta sección documenta el razonamiento detrás de cada una.

¿Por qué un **Foreground Service**?

Android gestiona la memoria de forma agresiva y puede matar cualquier proceso que lleve tiempo en segundo plano para liberar recursos. Un *Background Service* convencional sería terminado en cuestión de minutos en dispositivos modernos, lo que dejaría al agente sin capacidad de monitorizar la ubicación.

Un **Foreground Service**, al mostrar una notificación visible al usuario, tiene prioridad de proceso mucho mayor y raramente es eliminado por el sistema. El costo es la notificación persistente, que es también el mecanismo que informa al usuario de que el agente está operativo: una consecuencia de diseño que resulta ser beneficiosa para la transparencia hacia el usuario.

¿Por qué `goAsync()` en el `BroadcastReceiver`?

El sistema Android otorga a los `BroadcastReceiver` un máximo de **10 segundos** para ejecutarse antes de forzar su terminación. Guardar un registro en la base de datos Room es una operación de I/O que puede superar ese límite, especialmente en dispositivos lentos o cuando la base de datos está siendo accedida simultáneamente.

`goAsync()` extiende la vida útil del receiver más allá de los 10 segundos, devolviendo un `PendingResult` que se cierra explícitamente con `pendingResult.finish()` al completar la corrutina de escritura. Esto garantiza que el evento siempre queda guardado, independientemente del tiempo que tarde la operación de base de datos.

¿Por qué acción explícita en el `PendingIntent`?

A partir de Android 12 (API 31), Google bloqueó los `BroadcastReceiver` implícitos por razones de seguridad: una app no puede recibir broadcasts de otras apps a menos que declare explícitamente la

acción y marque el receiver como exportado. La API de Geofencing envía el evento desde el proceso de Google Play Services, que es externo a WANDA.

La solución requiere dos cambios coordinados: añadir la acción

`com.wanda.app.ACTION_GEOFENCE_EVENT` al Intent del PendingIntent, y declarar `exported="true"` en el receiver del `AndroidManifest.xml`. Si falta cualquiera de los dos, el broadcast llega al sistema pero el receiver nunca se activa.

¿Por qué `exported="true"` en `GeofenceBroadcastReceiver`?

Directamente relacionado con el punto anterior: aunque marcar un receiver como `exported="true"` va en contra del principio de mínimo privilegio, es un requisito obligatorio impuesto por la API de Geofencing de Google. La alternativa de usar un `PendingIntent` basado en un Service o una Activity en lugar de un Receiver existe pero es más costosa en recursos y no encaja con el patrón de manejo de eventos efímeros que requiere el Geofencing.

¿Por qué `BootReceiver`?

Las geovallas registradas en Google Play Services **se eliminan** cuando el dispositivo se reinicia. Si no se hiciera nada, el agente quedaría desactivado silenciosamente tras un apagado. El `BootReceiver` cierra este ciclo de vida: al recibir `BOOT_COMPLETED`, consulta el estado persistido en `SharedPreferences` y, si el rastreo estaba activo, relanza el servicio para volver a registrar la geovalla. El usuario experimenta continuidad total sin intervención.

¿Por qué `LiveData` en lugar de `Flow`?

Ambos son mecanismos reactivos válidos en Android. `LiveData` se eligió por su integración nativa con el ciclo de vida de los Fragments: deja de emitir automáticamente cuando la vista está en segundo plano y se reactiva cuando vuelve a primer plano, evitando actualizaciones innecesarias o crashes por intentar actualizar una vista destruida. Para un proyecto de esta escala, `LiveData` ofrece suficiente potencia con menor complejidad que los `Flows` de `Coroutines`.

14. Solución de problemas

Si WANDA no se comporta como se espera, la mayoría de los problemas tienen relación con permisos no concedidos o con el comportamiento del sistema operativo Android hacia los procesos en segundo plano. La siguiente tabla recoge los casos más frecuentes.

Problema	Causa más probable	Solución
No llegan notificaciones al entrar/salir	Permiso de ubicación en segundo plano no concedido	Ajustes → Apps → WANDA → Permisos → Ubicación → "Permitir todo el tiempo"
La notificación de entrada/salida tarda mucho	El GPS tiene deriva o el radio de geovalla es muy pequeño	Aumenta el radio a 150–300 m desde "Editar casa". El Geofencing de Google no es instantáneo: puede tardar 2-5 minutos en detectar la transición.
El rastreo no se reactiva tras reinicio	Permiso <code>RECEIVE_BOOT_COMPLETED</code> bloqueado o batería optimizada	Verifica el permiso en ajustes del sistema. Desactiva la optimización de batería para WANDA (Ajustes → Batería → Optimización → WANDA → No optimizar).
La geovalla no detecta la transición	Google Play Services desactualizado	Actualiza Google Play Services desde Google Play Store.
Error al establecer la ubicación GPS	GPS desactivado o señal insuficiente	Activa el GPS en modo "Alta precisión" (Ajustes → Ubicación → Modo). Sal al exterior para mejorar la señal.
La app no funciona en emulador	Geofencing requiere GPS real y Play Services completo	Usa un dispositivo físico Android con Google Play Services instalado.
Notificación persistente no aparece	Permiso <code>POST_NOTIFICATIONS</code> denegado	Concede el permiso de notificaciones desde Ajustes → Apps → WANDA → Notificaciones.
El rastreo se detiene solo después de un rato	El fabricante del dispositivo tiene restricciones agresivas de batería	En dispositivos Xiaomi/Huawei/Samsung, añade WANDA a la lista de apps permitidas en segundo plano (varía según fabricante y versión de MIUI/EMUI/One UI).

Nota: El comportamiento del Geofencing puede variar significativamente entre fabricantes. Los dispositivos con versiones modificadas de Android (Xiaomi MIUI, Huawei EMUI, Samsung One UI) suelen tener restricciones adicionales sobre procesos en segundo plano que pueden interferir con el servicio de rastreo. Consulta dontkillmyapp.com para instrucciones específicas por fabricante.

WANDA v1.0 · Sistemi ad Agenti · Android (API 26–34) · Kotlin + MVVM

Enrique Robeles · Juan Felix Noblejas Torroba · Alex Rodriguez · Bernardo Gómez